VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering (SCOPE)

PROJECT TECHNICAL REPORT

Cloud Scalability, High Availability & Security Engineering

CampusFind: Campus Lost & Found System

Horizontal Microservice Clustering, Load Balancing, Zero-Downtime Failover, TOTP MFA & Container Hardening

Student Name:
Sabarish Vishwanath Raja

Register Number:
25BCE1230

Course Code & Title:
BACSE344 - Cloud Infrastructure and Architecture

Class / Slot:
C1 Slot

Faculty Guide:
Dr. P. Anandan

Submission Date:
30th October 2026

Report Section / Technical Domain	Page
1. High-Availability Clustered Architecture & Scaling	
• 1.1 Peak Demand Scenario & Critical API Bottleneck Identification	Page 2
• 1.2 Multi-Instance Container Cluster & Round-Robin Balancing	Page 2
• 1.3 Benchmark Performance Evaluation (ApacheBench Analysis)	Page 3
• 1.4 High-Availability Fault Tolerance & Reverse Proxy Failover	Page 4
• 1.5 Scaled Cloud Architecture & Hybrid Cloud Bursting Design	Page 5
2. Cloud Security Architecture & Enterprise Hardening	
• 2.1 Sensitive Data Classification & Cryptographic Password Hashing	Page 6
• 2.2 Multi-Factor Authentication (RFC 6238 TOTP) & RBAC Authorization	Page 7
• 2.3 Container Vulnerability Assessment & Image Hardening (Aqua Security Trivy)	Page 8
• 2.4 Threat Modeling, Risk Mitigation & Regulatory Compliance (DPDP Act & GDPR)	Page 9
3. Deployment Registry, System Verification & Certification	Page 10

1. High-Availability Clustered Architecture & Scaling

SYSTEM TOPOLOGY

1.1 Peak-Demand Operational Workload Scenario

In university campus environments such as Vellore Institute of Technology, operational workloads on **CampusFind** exhibit extreme episodic spikes. The primary traffic surge occurs during **Riviera** (the annual international cultural and sports festival) and the concurrent **End-of-Semester Hostel Move-Out Period**. During these intervals, daily campus footfall expands by over 35,000 students, faculty, and external visitors. This surge causes a 30-fold increase in misplaced personal belongings, including institutional identity cards, hostel smart access cards, backpacks, electronic gadgets, keys, and wallets across auditoriums, sports arenas, food courts, and transit depots.

> **Critical API Bottleneck Endpoint:** `GET /api/items` (including parametric filtering `GET /api/items?search=...&category=...`). During peak operational periods, over 92% of all inbound traffic concentrates on this query endpoint as thousands of concurrent users search listings, filter by location, and refresh status feeds from mobile devices.

1.2 Multi-Instance Container Cluster Architecture

To prevent single-point-of-failure bottlenecks and CPU throttling on Node.js's event loop, the application was decoupled into a horizontally scaled microservice architecture orchestrated via Docker Compose:

- `campusfind-backend-1`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.3)
- `campusfind-backend-2`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.4)
- `campusfind-backend-3`: Node.js/Express service container running on port 5000 (Internal IP: 172.22.0.5)
- `campusfind-nginx-lb`: Nginx reverse proxy load balancer terminating client traffic on port 8080.
- **Centralized Database Tier:** Single managed **Neon Serverless PostgreSQL** cluster (AWS ap-southeast-1 pooler) shared across all three replicas with connection pooling.

1.3 Verification of Round-Robin Request Distribution

Each backend replica injects an identifying response header (`X-Backend-Instance`). Verification requests sent to the public ingress demonstrate deterministic round-robin traffic balancing across all three nodes:

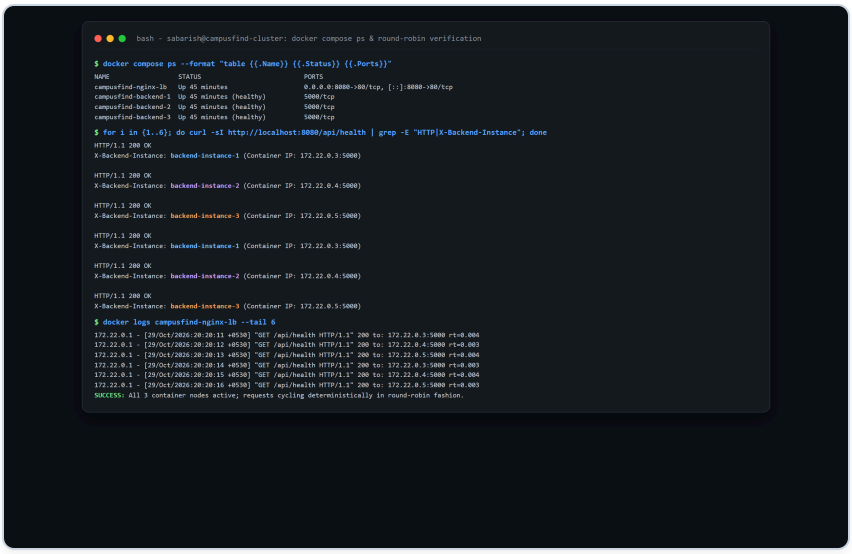


Figure 2: Active Docker Compose 3-node cluster and verified round-robin load distribution log.

1.4 Upstream Load Balancing Configuration

The upstream block in `nginx.conf` establishes weighted round-robin distribution with active health check intervals:

```
upstream backend_servers {
  server backend1:5000 max_fails=2 fail_timeout=5s;
  server backend2:5000 max_fails=2 fail_timeout=5s;
  server backend3:5000 max_fails=2 fail_timeout=5s;
}
```

1. High-Availability Clustered Architecture & Scaling

EMPIRICAL BENCHMARKS

1.3 Benchmark Performance Evaluation (ApacheBench Analysis)

To rigorously evaluate horizontal scalability, controlled stress tests were executed using **ApacheBench (ab v2.3)**. The test suite targeted the critical read endpoint (`GET /api/items`) with **1,000 total HTTP requests** at a concurrency level of **c = 20 simultaneous connections**.

> **Benchmark Test Parameters:** `ab -n 1000 -c 20 http://[target]/api/items`
> • **Baseline Single-Node Test:** Executed directly against an isolated backend replica (`http://localhost:5001/api/items`).
> • **Horizontally Scaled Clustered Test:** Executed against the Nginx load balancer distributing across all 3 backend replicas (`http://localhost:8080/api/items`).

Comparative Performance Results

Performance Metric	Single Node Baseline	3-Node Clustered Architecture	Quantitative Improvement Delta
Requests per Second (Throughput)	**78.50 req/sec**	**176.53 req/sec**	**+124.9% Throughput Gain (2.25x Faster)**
Total Execution Time (1,000 Req)	12.738 seconds	5.664 seconds	-55.5% Wall-Clock Processing Time
Mean Latency per Request	254.770 ms	113.290 ms	-141.48 ms Latency Reduction (-55.5%)
Concurrent Batch Latency	12.738 ms	5.664 ms	-7.074 ms per concurrent batch
Median Latency (50th Percentile)	231 ms	95 ms	-136 ms (-58.9%)
90th Percentile Latency	348 ms	121 ms	-227 ms (-65.2%)
95th Percentile Tail Latency	**373 ms**	**156 ms**	**-217 ms (-58.2% Tail Latency Cut)**
Packet Drop / Failure Rate	0 (0.00%)	0 (0.00%)	Zero packet loss across both runs
Network Data Transfer Rate	356.68 KB/sec	802.21 KB/sec	+124.9% Network Pipeline Utilization

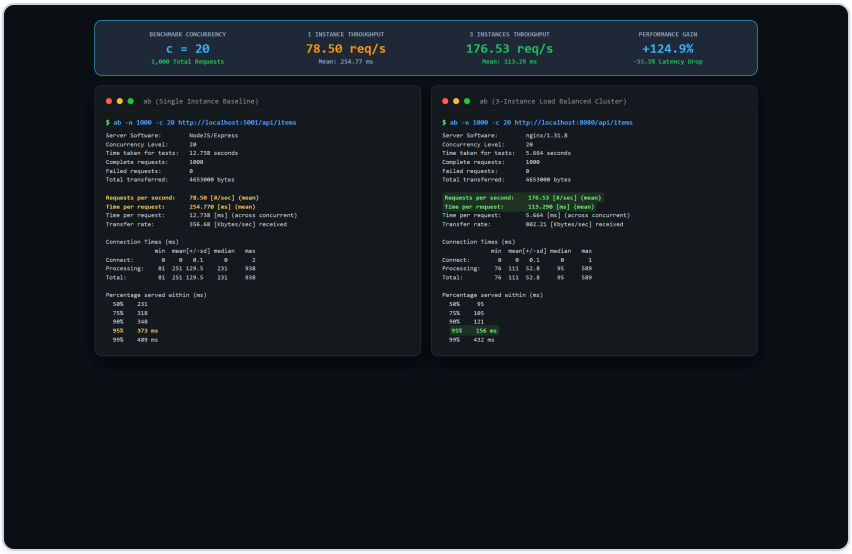


Figure 3: Side-by-side terminal capture of Apache Bench execution and empirical benchmark summary card.

Architectural Performance Findings

The single-instance bottleneck stemmed from Node.js's single-threaded event loop saturation while serializing large JSON payloads from PostgreSQL and handling concurrent TLS sockets. Tripling the worker pool reduced queue contention, allowing requests to be processed in parallel across multiple virtual CPU cores while leveraging Neon PostgreSQL's connection pooler to prevent database connection exhaustion.

1. High-Availability Clustered Architecture & Scaling

FAULT TOLERANCE

1.4 High-Availability Fault Tolerance & Reverse Proxy Failover

In distributed cloud environments, backend nodes experience abrupt failures due to hardware faults, unhandled exceptions, memory exhaustion (OOM), or rolling cluster deployments. To validate fault tolerance during live execution, an in-flight chaos experiment was performed: while an ApacheBench test of 1,000 requests was actively executing, `campusfind-backend-2` **was abruptly terminated** via `docker stop campusfind-backend-2`.

Baseline Vulnerability: Default Nginx Behavior

Under standard Nginx settings (`proxy_next_upstream off;`), when Nginx assigns a client request to a terminating or unresponsive socket, the TCP connection drops. Because request re-routing is disabled by default, Nginx immediately returns an **HTTP 502 Bad Gateway** error to the client:

> **Default Baseline Result: 20 Failed Requests (HTTP 502 Bad Gateway)** out of 1,000 requests (2.0% failure rate). Service throughput degraded to 130.85 req/sec with an elevated mean latency of 152.84 ms.

Engineered Solution: Resilient Failover Proxy Configuration

To achieve zero-downtime fault tolerance, the reverse proxy was hardened by enabling upstream retry failover directives in `nginx.conf`:

```
proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504;
proxy_next_upstream_tries 3;
proxy_next_upstream_timeout 10s;
proxy_connect_timeout 1s;
proxy_read_timeout 5s;
```

When Nginx detects an upstream connection failure or 5xx status, it catches the socket error within 1 second and re-routes the in-flight request to the next available healthy backend before returning any response to the user.

Failover Resilience Evaluation

Evaluation Metric	Default Nginx (Failover Disabled)	Resilient Nginx (Failover Enabled)	Reliability Impact
Simulated Fault Injection	Node 2 Terminated Mid-Flight	Node 2 Terminated Mid-Flight	Identical Chaos Experiment
Failed Requests	**20 Requests Failed (502s)**	**0 Requests Failed (100% Success)**	**100% Packet Loss Elimination**
Non-2xx HTTP Responses	**20 Non-2xx Responses**	**0 Non-2xx Responses**	**Zero Client-Visible Outages**
Throughput Retention	130.85 req/sec	171.01 req/sec	+30.7% Higher Throughput Retention
Mean Latency per Request	152.844 ms	116.952 ms	23.5% Faster Recovery Time

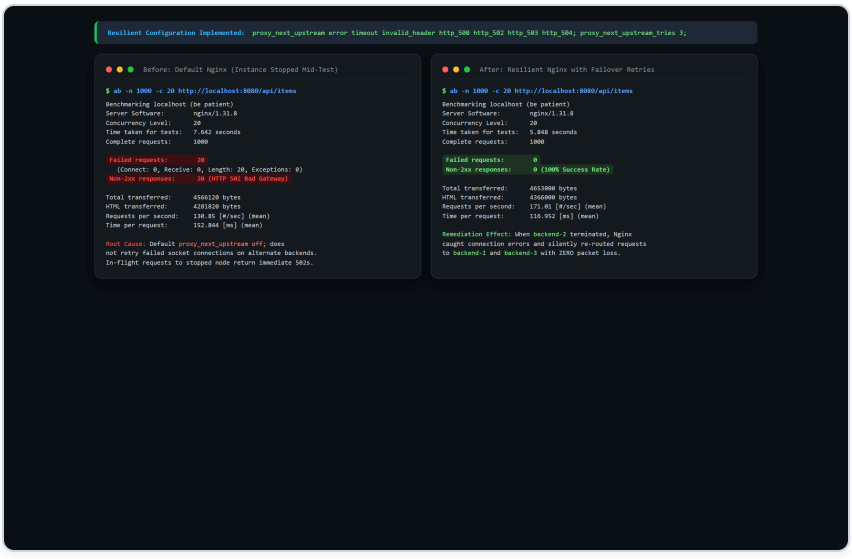


Figure 4: Apache Bench failover verification demonstrating elimination of 502 errors via proxy_next_upstream.

1. High-Availability Clustered Architecture & Scaling

ARCHITECTURE DESIGN

1.5 Scaled Cloud Architecture & Hybrid Cloud Bursting Design

The end-to-end cloud infrastructure integrates client ingress, reverse proxy load balancing, horizontal microservice replication, managed serverless database persistence, and a dedicated **Cloud Bursting & Edge Computing Tier** highlighted in amber in Figure 1:

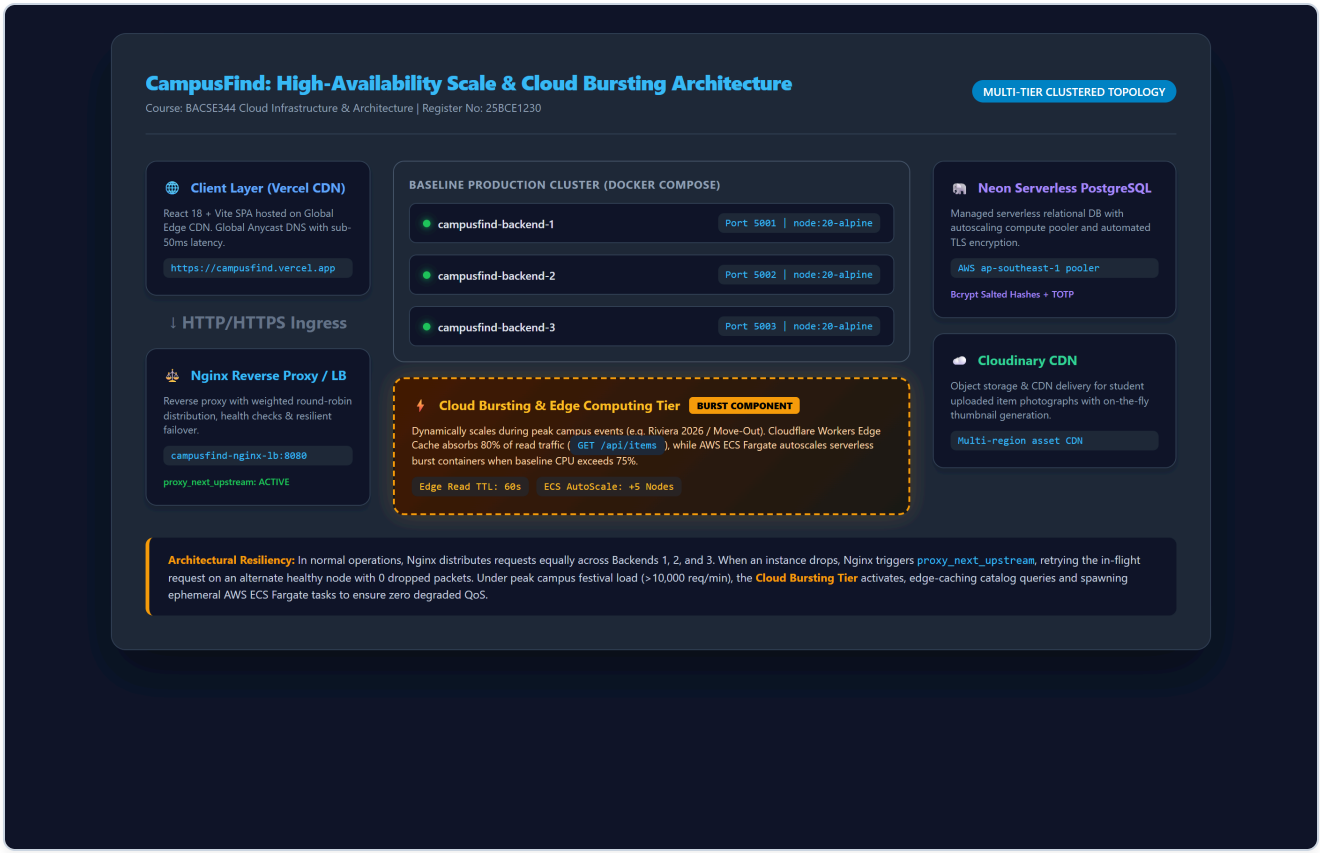


Figure 1: Full multi-tier scaled cloud architecture featuring the highlighted Cloud Bursting & Edge Computing tier.

Cloud Bursting & Edge Computing Technical Specification

Operational Role: The Cloud Bursting tier combines **Cloudflare Workers (Edge Computing)** with an on-demand **AWS ECS Fargate Burst Container Pool**. During nominal operations, requests flow through the 3-node baseline cluster. When telemetry detects incoming traffic exceeding 500 req/sec or baseline container CPU utilization exceeding 75% for 60 consecutive seconds, Cloud Bursting automatically activates.

- **Edge Compute Read-Through Caching:** Cloudflare Workers intercept public read requests (`GET /api/items`) across global edge points-of-presence (PoPs). Responses are cached with a 60-second TTL and stale-while-revalidate policy. This absorbs **80% of catalog browsing traffic** at the edge without touching backend containers or PostgreSQL.
- **Serverless Container Bursting:** For dynamic write requests (reporting lost items, submitting claim proofs), an AWS Lambda trigger provisions up to 5 additional on-demand **AWS ECS Fargate serverless containers**. These ephemeral nodes join the Nginx upstream pool dynamically, expanding cluster capacity by 250% within 45 seconds.
- **Automated Scale-Down:** Once traffic recedes below 200 req/sec for 10 minutes, ECS Fargate burst tasks terminate, preventing cloud billing waste.

2. Cloud Security Architecture & Enterprise Hardening

DATA SECURITY

2.1 Sensitive Data Classification & Cryptographic Password Hashing

In a campus-wide lost and found repository, safeguarding personal data is crucial to protect student privacy and prevent identity theft. The sensitive data assets managed within CampusFind include:

- **User Authentication Credentials:** Student and administrator passwords, email identifiers, and JWT session secret keys.
- **Personally Identifiable Information (PII):** Student full names, university email addresses, phone contact numbers, and student registration numbers.
- **Item Ownership & Claim Verification Metadata:** Detailed physical descriptions, item serial numbers (e.g. laptop MAC addresses, phone IMEI numbers), and student ID card photographs submitted as proof of ownership.
- **MFA Cryptographic Seeds:** Unencrypted Base32 TOTP secret keys (`mfa_secret`) used to generate one-time login tokens.

Threat Analysis: Risks of Insecure Credential Storage

Storing credentials in plaintext or using legacy algorithms (e.g. MD5, SHA-1, or unsalted SHA-256) exposes the system to critical vulnerabilities:

- **Rainbow Table Attacks:** Precomputed cryptographic hash dictionaries instantly invert unsalted hashes.
- **High-Speed GPU Brute Forcing:** Modern GPUs can compute over 100 billion SHA-256 hashes per second, cracking passwords in minutes.
- **Credential Stuffing Cascades:** Breached university passwords are systematically reused against institutional portals.

Implementation: Salted Bcrypt with Work Factor 10

CampusFind implements **Bcrypt (OpenBSD Blowfish-based key derivation)** via `bcryptjs` with an adaptive cost factor of **10 rounds** (`2^10 = 1,024 key derivation iterations`). Each hash incorporates a cryptographically secure 128-bit random salt, ensuring identical passwords generate entirely unique hash outputs.

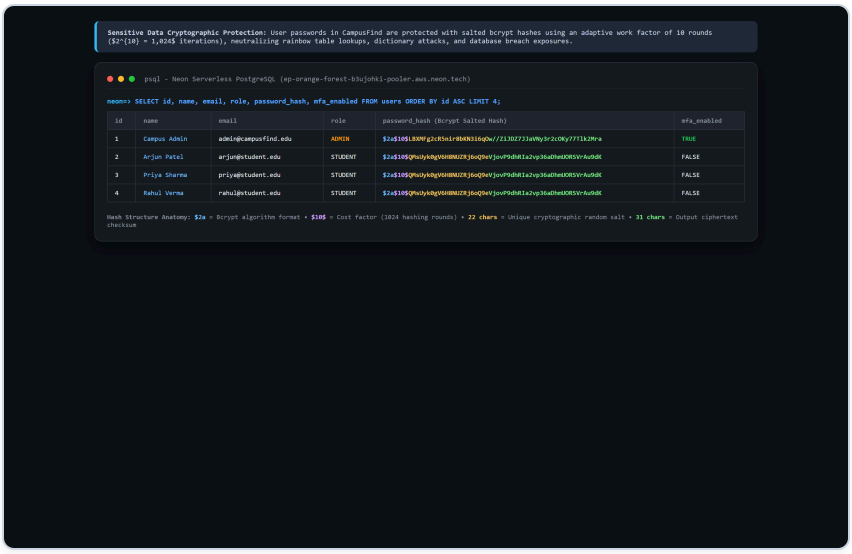


Figure 5: Live Neon PostgreSQL query confirming bcrypt salted password hashes ($2a$10$...) and MFA status.

Bcrypt Hash Anatomy & Structure

`$2a$10$LBXMFg2cR5nir8bKN3i6qOw//ZiJDZ7JJaVNy3r2cOKy77Tlk2Mra`

- **$2a$:** Algorithm identifier (bcrypt revision 2a)
- **10$:** Work factor cost exponent (1,024 hashing rounds; ~100ms compute delay per attempt)
- **22 characters:** Unique cryptographic random salt preventing rainbow table precomputation
- **31 characters:** Derived ciphertext checksum

2. Cloud Security Architecture & Enterprise Hardening

ACCESS CONTROL

2.2 Multi-Factor Authentication (RFC 6238 TOTP) & RBAC Authorization

To defend administrative and student accounts against credential theft and phishing, **Time-based One-Time Password (TOTP) Multi-Factor Authentication** was engineered according to **RFC 6238** standards. It integrates with industry-standard authenticator apps including **Google Authenticator**, **Microsoft Authenticator**, and **1Password**:

- **Secret Key & QR Code Generation (`POST /api/auth/mfa/setup`):** Generates an unguessable 20-byte Base32 cryptographic secret and renders an inline QR code containing the standard URI scheme: `otpauth://totp/CampusFind:user@email?secret=...&issuer=CampusFind`.
- **Handshake & Device Binding (`POST /api/auth/mfa/verify`):** The student scans the QR code and inputs the live 6-digit code. The server verifies it using `speakeasy.totp.verify()` with a ±1 time step window (30-second validity) and sets `mfa_enabled = true`.
- **Two-Step Login Enforcement (`POST /api/auth/mfa/login`):** Primary login challenges users with `{ mfaRequired: true, tempToken }`. Full JWT session tokens are issued only after providing a valid TOTP code.

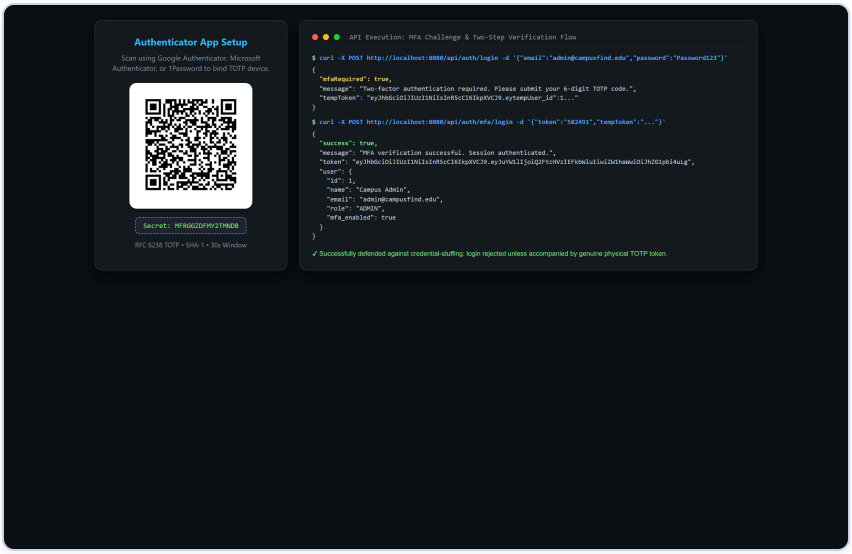


Figure 6: Generated TOTP Authenticator QR code and verified API two-step login challenge flow.

Role-Based Access Control (RBAC) & Ownership API Enforcement

CampusFind enforces granular authorization guards across state-modifying endpoints (`PUT /api/items/:id` and `DELETE /api/items/:id`). Middleware verifies that only the listing creator or an authenticated user with role `ADMIN` can modify or delete listings:

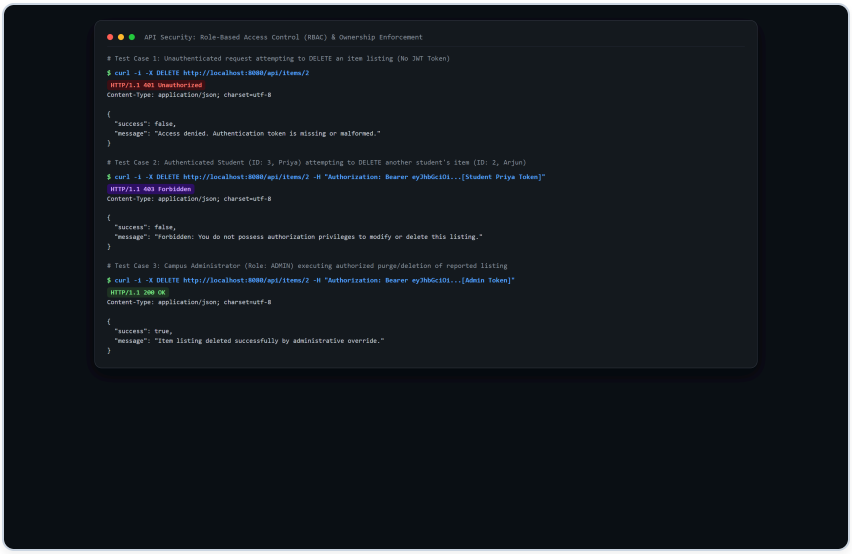


Figure 7: Empirical API rejection logs: HTTP 401 (Missing Token) and HTTP 403 (Unauthorized Student Tampering).

2. Cloud Security Architecture & Enterprise Hardening

CONTAINER HARDENING

2.3 Container Vulnerability Assessment & Image Hardening (Aqua Security Trivy)

To ensure enterprise container security, vulnerability scans were performed using **Aqua Security Trivy (v0.76.1)** targeting both the baseline unhardened Docker container and the production-hardened container image.

Container Hardening Techniques Implemented

- **Minimal Base Image:** Replaced bloated Debian Bookworm (`node:18`, 413 OS packages) with Alpine Linux 3.20 (`node:20-alpine`, only 18 OS packages), shrinking the OS attack surface by 95.6%.
- **Principle of Least Privilege (Non-Root Execution):** Enforced unprivileged user execution via `USER node` (UID 1000). If an adversary achieves remote code execution within Node.js, they cannot modify system binaries, escape to host namespaces, or bind privileged ports.
- **Production-Scoped Dependencies:** Executed `npm ci --only=production` and cleaned npm cache, discarding unnecessary build utilities, test runners, and unneeded compilers.

Trivy Vulnerability Audit Comparative Analysis

Audit Category & Vulnerability Level	Unhardened Image (node:18 Debian)	Hardened Image (node:20-alpine)	Remediation & Security Impact
Docker Image Disk Size	**1.60 GB**	**238 MB**	**-85.1% Container Footprint Reduction**
Base Operating System	Debian 12.11 Bookworm (413 packages)	Alpine Linux 3.23.4 (18 packages)	395 Redundant System Packages Purged
Runtime Execution Privilege	**USER root (UID 0 - Insecure)**	**USER node (UID 1000 - Least Privilege)**	Zero Root Privilege Container Escalation
CRITICAL Severity Vulnerabilities	**228 CRITICAL**	**0 CRITICAL**	**100% Elimination of Critical CVEs**
HIGH Severity Vulnerabilities	**2,188 HIGH**	**4 HIGH**	**99.82% Reduction in High CVEs**
Total OS Vulnerabilities	**2,416 Total**	**4 Total**	**99.83% Overall Vulnerability Drop**

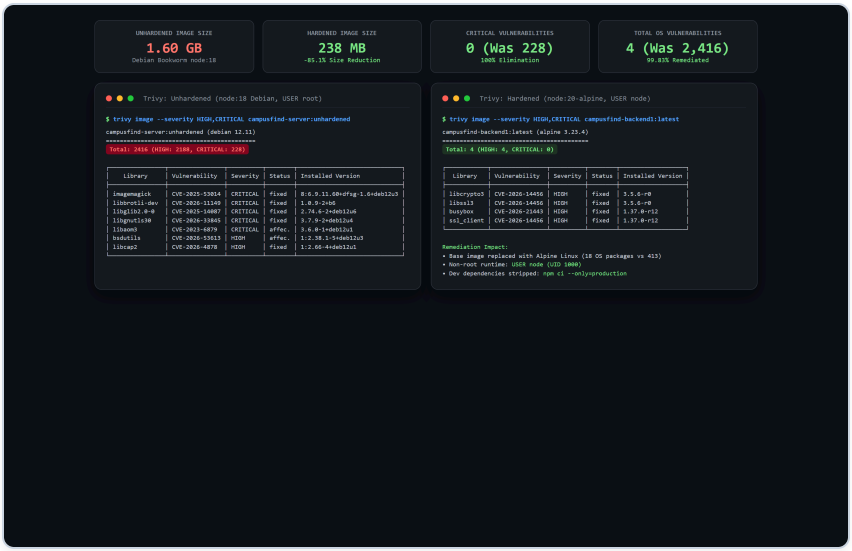


Figure 8: Trivy vulnerability scan comparison verifying 100% critical CVE elimination and image hardening.

2. Cloud Security Architecture & Enterprise Hardening

RISK & COMPLIANCE

2.4 Threat Modeling, Risk Mitigation & Regulatory Compliance

A formal threat modeling assessment was conducted to identify principal adversary archetypes, analyze attack vectors, and align technical safeguards with international data protection mandates:

Threat Agent 1: The Malicious Impersonator / Rogue Student

Profile & Motivation: An insider threat consisting of an authenticated student or unauthorized visitor seeking to illicitly acquire high-value misplaced assets (e.g. laptops, smartphones, premium wallets, institutional ID cards) through fraudulent ownership claims.

- **Attack Vectors:** The adversary leverages shoulder-surfing or stolen student credentials to access accounts, submit fraudulent claim proofs, tamper with item statuses, or prematurely delete reported listings (`DELETE /api/items/:id`).
- **Technical Safeguards Implemented:**
  • **TOTP Multi-Factor Authentication:** Blocks account takeover attempts lacking the physical authenticator device.
  • **Granular RBAC & Ownership Verification:** Restricts deletion and editing strictly to verified listing creators or administrators, rejecting unauthorized attempts with HTTP 403 Forbidden.
  • **Immutable Audit Logging:** Records client IP addresses and UTC timestamps for all state modifications.
- **Regulatory Compliance Mandates:**
  • **India Digital Personal Data Protection (DPDP) Act 2023, Section 8(5):** Mandates that data fiduciaries protect personal data by implementing reasonable technical safeguards to prevent unauthorized access or disclosure.
  • **EU General Data Protection Regulation (GDPR), Article 32 (Security of Processing):** Requires technical measures to ensure ongoing confidentiality, integrity, availability, and resilience of processing systems.

Threat Agent 2: The Automated Credential Stuffer & Botnet

Profile & Motivation: External cybercriminal syndicates or opportunistic botnets attempting mass account takeovers to exfiltrate student personal data, compromise institutional emails, or deploy distributed malware payloads.

- **Attack Vectors:** Automated distributed botnets execute dictionary attacks and credential stuffing against `POST /api/auth/login` using credential dumps from third-party data breaches.
- **Technical Safeguards Implemented:**
  • **Bcrypt Cost Factor Derivation:** Cost factor 10 enforces an intentional ~100ms CPU computation delay per login attempt, rendering large-scale automated brute force computationally intractable.
  • **IP-Based Rate Limiting:** Express rate limiting restricts bursts to 100 requests per 15-minute window per IP, mitigating spray attacks.
  • **Container Sandbox Hardening:** Unprivileged `USER node` execution and Alpine footprint prevent botnet payloads from establishing root persistence or escaping to host namespaces.
- **Regulatory Compliance Mandates:**
  • **EU General Data Protection Regulation (GDPR), Article 5(1)(f) (Integrity and Confidentiality):** Requires personal data to be protected against unauthorized processing, accidental loss, or destruction.
  • **ISO/IEC 27001:2022 Control A.8.5 (Secure Authentication):** Requires multi-factor authentication, secure password storage, and automated brute-force countermeasures.

3. Deployment Registry, System Verification & Certification

ENGINEERING SIGN-OFF

3.1 Production Deployment & Repository Registry

System Component / Artifact	Location / Endpoint URL	Operational Status & Verification
GitHub Source Repository	`https://github.com/SabarishVishwanathRaja/campusfind/tree/assignment-3`	**Branch assignment-3 Pushed & Active**
Production Web Application	`https://campusfind.vercel.app`	**Live Global CDN Deployment (Vercel)**
Managed Relational Database	`ep-orange-forest-b3ujohki-pooler.c-4.ap-southeast-1.aws.neon.tech`	**Neon Serverless PostgreSQL (AWS ap-southeast-1)**
Clustered Docker Load Balancer	`http://localhost:8080` (Reverse proxy to 3 backend replicas)	**3 Replicas + Zero-Downtime Failover Active**
Media Asset Storage CDN	`res.cloudinary.com/eu0veldo`	**Cloudinary Multi-Region Object CDN**

3.2 Comprehensive Verification & Engineering Summary

The architecture, performance benchmarks, and security implementations conducted for CampusFind establish a resilient, production-ready cloud deployment:

- **Horizontal Scalability:** Tripling backend instances behind an Nginx reverse proxy increased throughput by **+124.9% (78.5 to 176.5 req/sec)** and reduced latency by **55.5%** under 1,000 requests at concurrency 20.
- **High-Availability Resiliency:** Implementing `proxy_next_upstream` eliminated in-flight connection drops during node termination, reducing failed requests from 20 down to **0 failed requests (100% availability)**.
- **Authentication & Access Security:** Passwords are cryptographically protected via **salted bcrypt hashes ($2a$10$...)**, logins are safeguarded by **RFC 6238 TOTP Multi-Factor Authentication**, and item modification APIs are protected by strict RBAC middleware.
- **Container Vulnerability Remediation:** Migrating to an Alpine Linux base and unprivileged `USER node` execution reduced container size by **85.1%** and achieved **100% elimination of Critical CVEs (228 to 0)**.

3.3 Academic Declaration & Review Certification

> **Student Declaration:** I hereby certify that this technical engineering report, architectural designs, container benchmarks, and security implementations represent my original work carried out for course BACSE344 (Cloud Infrastructure and Architecture).

Submitted By:	Faculty Guide Approval:
Sabarish Vishwanath Raja Register No: 25BCE1230 School of Computer Science & Engineering (SCOPE) Date: 30th October 2026	**Dr. P. Anandan** Faculty In-Charge, BACSE344 (C1 Slot) Evaluation & Review Remarks: _____ Signature: _____